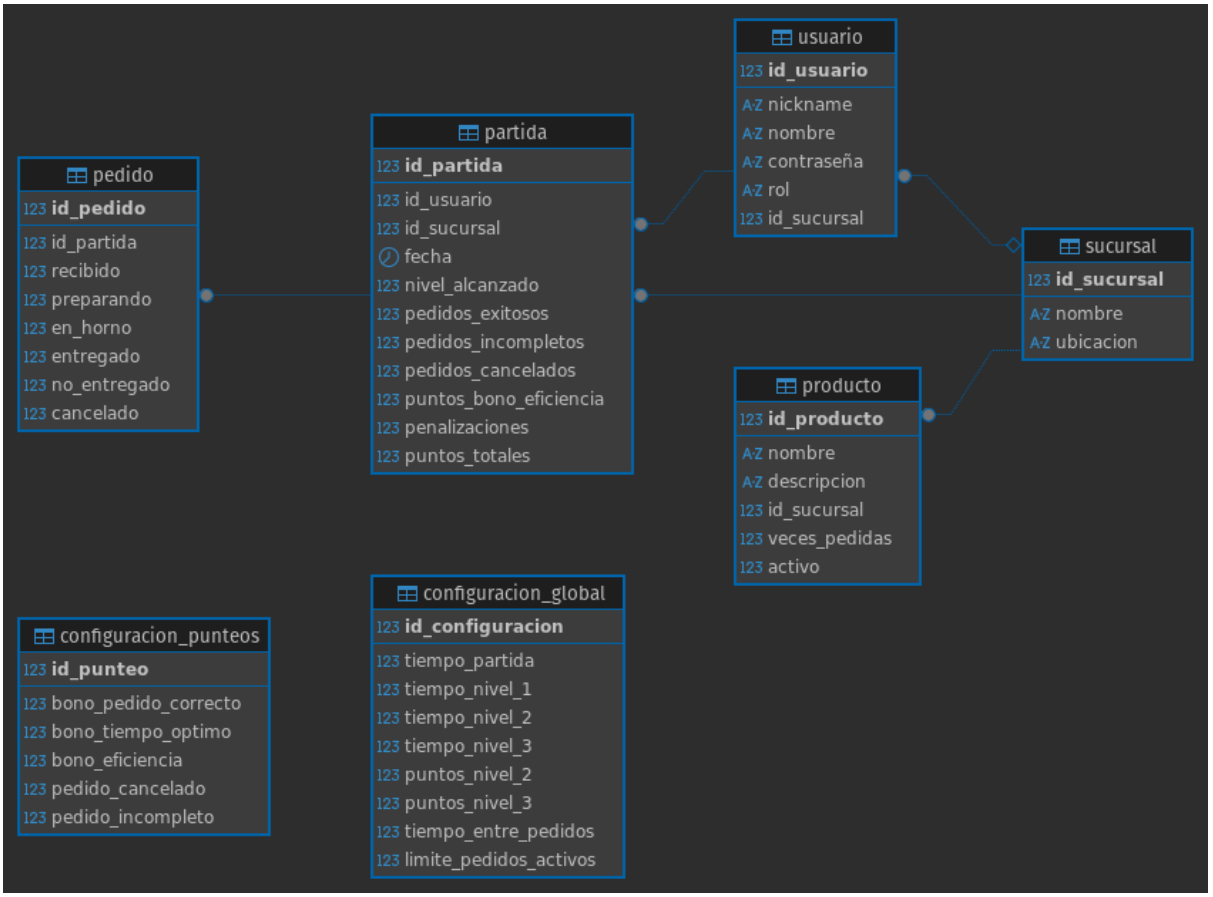
MANUAL TECNICO PIZZA TYCON

La práctica fué realizada usando java 21, para el backend, java swing para el frontend y mysql como gestor de base de datos.

Se usó NetBeans como editor de código, Linux Pop Os 22 como sistema operativo

Base de datos

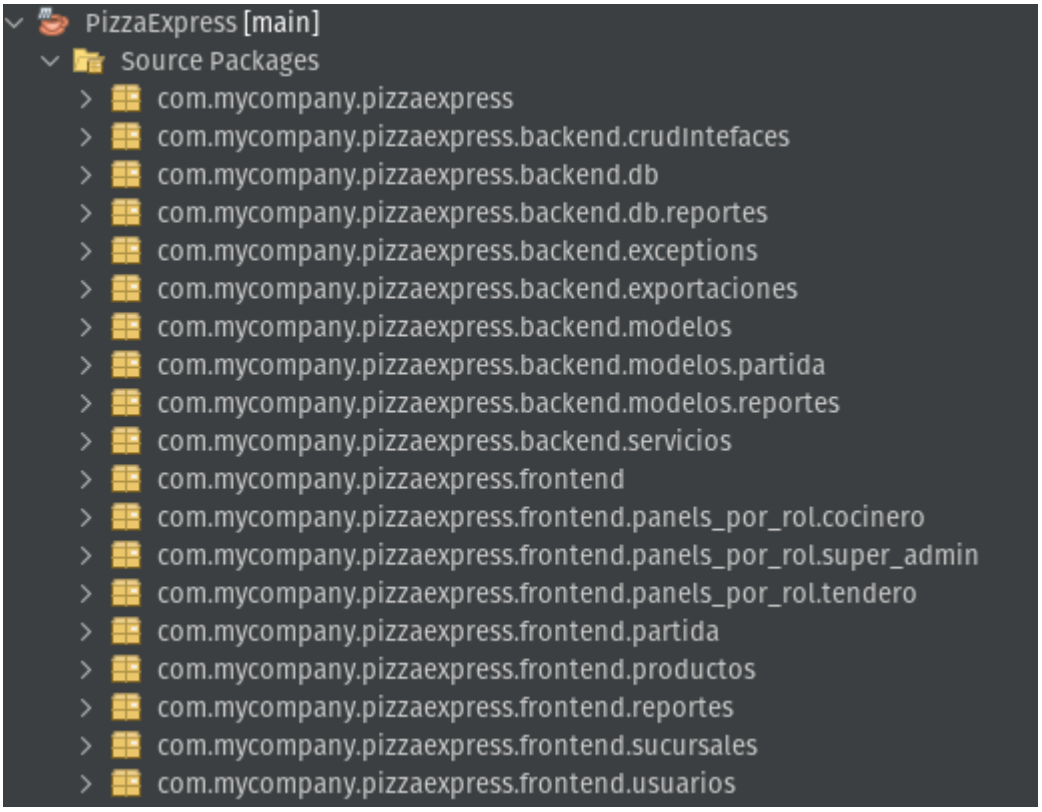
Al principio se pensó la arquitectura de la base de datos con 9 entidades pero conforme se avanzó se notó que no se necesitaban tantas, quedando el diagrama de tablas de la siguiente manera:



Las entidades que se crearón y puede se eliminaron fueron: producto pedido, tabla estados, porque realmente se agregó un atributo a producto que cuenta cuántas veces ha sido usado.

Como detalle, se implementó el ON DELETE CASCADE si elimina una sucursal, se eliminan todos los registros asociados

Ordenamiento de paquetes código fuente



En el código fuente hay dos paquetes principales el frontend y el backend,

Paquetes más importantes:

Backend:
modelos, db ,servicios, reportes

Frontend:
paneles_por_rol
partida
productos
sucursales
usuarios
reportes

Flujo de un CRUD de una entidad

Para crear un crud se usaron 4 elementos

1. **Clase modelo :** es la que tiene toda la información de la entidad a crear con getters, setters, y constructores, es la que se usa desde la interfaz para pasarla como parametro al backend
2. **Clase db :** esta clase se encarga del crud, no realiza verificaciones sino que solo se encargar crear, eliminar, editar o leer la entidad deseada usando la clase modelo.
3. **CRUD service:** esta clase se encarga de llamar a cada método de la clase db de entidad para realizar el crud, pero aquí es donde suceden las verificaciones, es un intermediario entre el frontend y la clase que solo realiza el crud

Ejemplo de un Crud (Sucursal)

Para poder crear una sucursal primero se debe llenar desde el frontend la clase modelo

```
public class Sucursal {

    private String ubicacion;
    private String nombre;
    private int id;

    public Sucursal(String ubicacion, String nombre) {
        this.ubicacion = ubicacion;
        this.nombre = nombre;
    }

    public Sucursal(String ubicacion, String nombre, int id) {
        this.ubicacion = ubicacion;
        this.nombre = nombre;
        this.id = id;
    }
```

debe tener dos conductores con una diferencia de id, porque cuando se trata de buscar una sucursal para editar se debe devolver el id

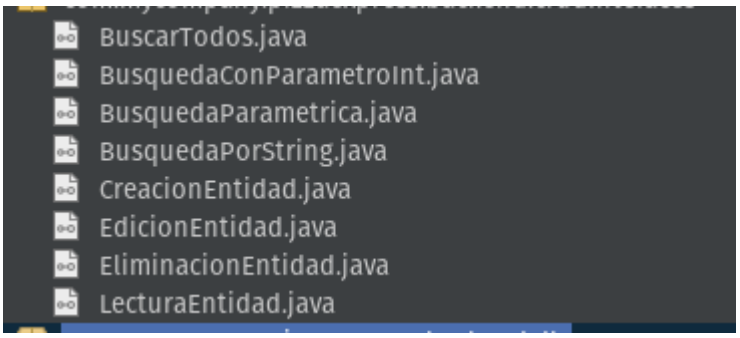
El siguiente paso es enviarla crud service que implementa diferentes interfaces para poder realizar el crud, se tomó la decisión de usar interfaces porque no todas las entidades necesitaban de un crud completo, por ejemplo partidas solo se debe crear y leer en este caso implementa las siguentes:

```
public class SucursalesCrudService implements CreacionEntidad<Sucursal>, EdicionEntidad<Sucursal>, EliminacionEntidad,
        BuscarTodos<Sucursal>, LecturaEntidad<Sucursal> {
```

Todas las interfaces:

```
public interface BuscarTodos <T>{
    public ArrayList<T> seleccionarTodos() throws ExceptionGenerica;
}


public interface BusquedaConParametroInt <T>{

    public ArrayList<T> buscarConUnParametroInt(int param) throws ExceptionGeneric
}

public interface BusquedaParametrica <B,A> {

     public ArrayList<A> buscarVariosConFiltro(B busqueda) throws ExceptionGeneric

}
public interface BusquedaPorString <T> {
    public ArrayList<T> buscarPorString(String parametro) throws ExceptionGenerica
}

public interface CreacionEntidad <T>{
    public void crear(T entidad) throws ExceptionGenerica;
}

public interface EdicionEntidad <T>{
    public void  editar(T entidad) throws ExceptionGenerica;
}
public interface EliminacionEntidad {
    public void eliminar(int id) throws ExceptionGenerica;

}

public interface LecturaEntidad <T>{
    public  T buscar(int id) throws ExceptionGenerica;
}
```

En total se usaron 9 interfaces diferentes



La clase crud service, realiza las validaciones necesarias un ejemplo es la validación par evitar sucursales duplicadas:

```java
@Override
public void crear(Sucursal entidad) throws ExceptionGenerica {
    if (!entidad.datosCompletos()) {
        throw new CamposVaciosException();
    }
    if (!entidad.esValido()) {
        throw new DatosMuyLargosException();
    }

    if (db.yaExiste(entidad.getNombre())) {
        throw new EntidadDuplicadaException("ya existe la sucursal con nombre " + entidad.getNombre());
    }

    db.crear(entidad);
}
```

y luego manda a llamar a la clase db en su método crear

```java
@Override
public void crear(Sucursal entidad) throws ExceptionGenerica {
    try (Connection conn = ConexionDB.getConnection(); PreparedStatement stmt = conn.prepareStatement(CREAR)) {
        stmt.setString(1, entidad.getNombre());
        stmt.setString(2, entidad.getUbicacion());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new ExceptionGenerica("Error al crear sucursal: " + e.getMessage());
    }
}
```

así es el flujo para crear una entidad, lo mismo pasa con editar, eliminar (en ese caso solo se necesita del id), y leer algunos métodos devuelven arrayList o simplemente un objeto dependiendo del caso.

Clases necesarias para una partida y lógica de partida

Cuando se decide iniciar una partida se instancia un generador de partidas, no sin antes validar que hayan productos existentes en la sucursal, la verificación se realiza con la clase productos DB

```java
public PanelPartida(CocineroPanelBase panelPadre) {
    initComponents();
    this.convertidor = new ConvertidorDeTiempo();
    this.panelPadre = panelPadre;
    this.generadorPartidas = new GeneradorPartidas();

    this.panelDetalles.add(new PanelBaseDetalle(), BorderLayout.CENTER);
    this.repaint();
    this.revalidate();

    try {
        partida = generadorPartidas.generarNuevaPartida();
        this.organizarPanelPedidos(partida.getLimitePedidosActivos());
        partida.setPanelPartida(this);
        partidaHilo = new Thread(partida);
        partidaHilo.start();
    } catch (ExceptionGenerica ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage());
    }
}
```

```java
public class GeneradorPartidas {
    private ConfiguracionesCrudService crudService = new ConfiguracionesCrudService();

    public Partida generarNuevaPartida() throws ExceptionGenerica{
        return new Partida(crudService.obtenerReglasPartida(), crudService.obtenerReglasPunteos());
    }
}
```

este devuelve una partida con las configuraciones previamente consultadas a la db, esto se realiza en el panel partida, que después de recibir la partida, inicia un hilo. La partida conoce al panel del frontend con lo cual puede actualizar su información conforme sea necesario

La clase partida implementa la interfaz runnable y en su método run instancia un reloj, instancia el generador de partidas e inicia ambos hilos

```java
public Partida(ConfiguracionDePartida reglasPartida, ConfiguracionPunteos reglasPunteo) {
    this.reglasPartida = reglasPartida;
    this.reglasPunteo = reglasPunteo;
    this.idUsuario = Sesion.getUsuario().getId();
    this.idSucursal = Sesion.getUsuario().getIdSucursal();
}

@Override
public void run() {
    partidaEnCurso = true;
    reloj = new Reloj(reglasPartida.getTiempoPartida(), this);
    hiloReloj = new Thread(reloj);
    generadorPedidos = new GeneradorDePedidos(hiloReloj, this, reglasPartida);
    hiloReloj.start();
    Thread hiloGenerador = new Thread(generadorPedidos);
    hiloGenerador.start();
}
```

Reloj

la clase reloj implementa la interfaz runnable y conoce a la partida
su trabajo es ir disminuyendo el tiempo a la vez que le dice a la partida que actualice su frontend, además es quien controla todo, porque una vez terminado el hilo, pues pone la partida Curso a false

```java
public class Reloj implements Runnable {

    private volatile int tiempoRestante;
    private Partida partida;

    public Reloj(int tiempoRestante, Partida partida) {
        this.partida = partida;
        this.tiempoRestante = tiempoRestante;
    }

    @Override
    public void run() {
        while (tiempoRestante > 0) {
            try {
                tiempoRestante -= 1000;
                Thread.sleep(1000);
                partida.actualizarTiempoFrontend(tiempoRestante);

            } catch (InterruptedException ex) {
                Thread.currentThread().interrupt();
                break;
            }
        }
        this.partida.setPartidaEnCurso(false);
        this.partida.avisarPartidaTerminada();
    }

    public int getTiempoRestante() {
        return tiempoRestante;
    }

    public void acabarTiempo() {
        this.tiempoRestante = 0;
    }

}
```

Generador de pedidos

es una clase que implementa el método run en el cual tomando en cuenta el tiempo entre pedidos configurado, duerme el hilo el tiempo especificado, y luego verifica que la partida siga viva, si está viva entonces intentar agregar un pedido a la partida, según el nivel y si hay espacio.

```java
@Override
public void run() {
    while (relojPartida.isAlive() && partida.isPartidaEnCurso()) {
    try {
        intentarGenerarPedido();
        Thread.sleep(tiempoEntrePedidos);
    } catch (InterruptedException ex) {
        return;
    } catch (ExceptionGenerica ex) {
        partida.mostrarErrorEnPedido(ex.getMessage());
    }
}
}
```

Agregación de pedido a partida

la partida tiene un método para agregar un pedido el cual lo guarda en una lista y le pide al frontend que agregue un pedido

```java
public void agregarNuevoPedido(Pedido pedido) {
    //System.out.println("Agregó #"+pedido.getNumeroPedido());
    pedidosActivos++;
    numeroPedidos++;
    pedido.setNumeroPedido(numeroPedidos);
    this.listaPedidos.add(pedido);
    this.panelPartida.agregarNuevoPedido(pedido);
    this.panelPartida.actualizarEstadisticas();
}
```

Panel pedido

EL panel pedido representa un pedido, recibe como parámetro un pedido e inicia un nuevo hilo

```java
public PanelPedido(Pedido pedido, JPanel panelContenedor, PanelPartida panelPartida) {
    initComponents();
    this.panelPartida = panelPartida;
    this.convertidorDeTiempo= new ConvertidorDeTiempo();

    this.setSize(new Dimension(293, 246));
    this.pedido = pedido;
    this.panelContenedor = panelContenedor;
    this.numeroPedido.setText("No." + pedido.getNumeroPedido());
    Thread pedidoHilo = new Thread(pedido);
    actualizarEstado();
    pedido.setPanelPedido(this);
    pedidoHilo.start();
}
```

Clase pedido

Implementa la interfaz runnable , funciona como su propio reloj, al terminar el hilo le pide a la partida que lo contabilice

```java
@Override
public void run() {
    while (pedidoEstaVivo()) {
        //System.out.println("No."+ this.getNumeroPedido()+ "vivo");
        try {
            Thread.sleep(1000);
            tiempoRestante -= 1000;
            this.panelPedido.actualizarTiempo(tiempoRestante);
        } catch (InterruptedException ex) {
        }
    }

    //si no fue cancelado o entregado
    if ((!cancelado && !entregado)) {
        this.noEntregado = true;
    }
    //avisar al panel que ya terminó
    this.panelPedido.eliminarPedidoVisual();

    //avisar a la partida que ya terminó
    partida.ProcesarPedidoFinalizado(this);
}
```

tiene metodos para avanzar su estad

```java
public void avanzar() {
    if (ultimoEstado < 4) {
        ultimoEstado++;
        switch (ultimoEstado) {
            case 2:
                preparando = true;
                break;
            case 3:
                enHorno = true;
                break;
            case 4:
                entregado = true;
                activo = false; // termina el pedido
                break;
        }
    }
}
```

Contabilizar un pedido:

este método se encuentra en la partida y es llamado cada vez que un hilo de pedido termina lo que hace es contabilizar todos los puntos
y actualizar en el frontend las estadísticas

```java
public void ProcesarPedidoFinalizado(Pedido pedido) {
    //System.out.println("Contabilizado #"+pedido.getNumeroPedido());
    pedidosContados++;
    pedidosActivos--;
    if (pedido.isCancelado()) {
        pedidosCancelados++;
        penalizaciones += reglasPunteo.getPedidoCancelado();
    } else if (pedido.isNoEntregado()) {
        pedidosIncompletos++;
        penalizaciones += reglasPunteo.getPedidoIncompleto();
    } else if (pedido.isEntregado()) {
        puntosPositivos += reglasPunteo.getBonoPedidoCorrecto();
        pedidosExitosos++;
    }

    if (pedido.aplicaBonoAntesDeTiempo()) {
        puntosPositivos += reglasPunteo.getBonoTiempoOptimo();
    }
    if (pedido.aplicaBonoEficiencia()) {
        bonosPorEficiencia += reglasPunteo.getBonoEficiencia();
        puntosPositivos += reglasPunteo.getBonoEficiencia();
    }

    this.puntosTotales = puntosPositivos - penalizaciones;
    this.actualizarNivel();
    this.panelPartida.actualizarEstadisticas();
}
```

Gestión de punteos:

cada vez que termina un pedido, también manda a llamar a actualizar nivel, este con la ayuda de la tabla de configuración actualiza el nivel actual

```java
    /**
     * Actualiza el nivel segun los puntos actuales
     */
    public void actualizarNivel() {
        if (puntosTotales >= reglasPartida.getPuntosNivel2() && puntosTotales < reglasPartida.getPun
            this.nivelActual = 2;
        } else if (puntosTotales >= reglasPartida.getPuntosNivel3()) {
            this.nivelActual = 3;
        }
    }
```

Fin de partida:

cuando una partida finaliza, le avisa al frontend para moverse al panel de fin de partida, este panel tiene un timer para esperar a que se contabilicen todos los pedidos porque a veces pasa muy rápido y todas las estadísticas quedan en cero, es por eso que espera 2 segundos y medio.
Además este panel es el encargado de llamar a que se guarde la partida, y que se guarden las veces de productos.

```
private void guardarPartidaYProductos() {
    try {
        this.productosDB.contarProductosEnPedidos(partida.getListaPedidos());
        this.reportesPartidas.crear(partida);
    } catch (ExceptionGenerica ex) {
        JOptionPane.showMessageDialog(this, ex.getMessage());
    }

}
```